

Project 3 Report

2022059952 박준서

Design



Design

Explain your implementation plan to meet the assignment requirements.

먼저 `kernel/proc.h` 의 `proc` 구조체에 추가된 `vmass` 배열의 용도와 `mmap` 함수의 목적에 대해 살펴보았다.

각 프로세스 안에는 VMA (Virtual Memory Area)가 존재하며, `proc.h` 파일에 정의된 `vma` 구조체는 해당 slot에 어떤 파일(`struct file* f`)의 어느 범위가(`addr`, `length`) 어떤 권한/모드로(`prot`, `flags`) 매핑되어야 할지를 결정하는 변수들이 있다. `offset` 은 매핑이 파일의 어느 위치에서 시작하는지를 나타내며, 본 과제에서는 항상 0이다.

```
// mmap virtual memory area. One slot per active mapping.
#define NVMA 16

struct vma {
    int        used;        // 1 if this slot is in use
    uint64     addr;        // start virtual address (page-aligned)
    uint64     length;      // length in bytes (PGSIZE multiple)
    int        prot;        // PROT_READ / PROT_WRITE
    int        flags;       // MAP_SHARED / MAP_PRIVATE
    int        offset;      // file offset (always 0 in this lab)
    struct file *f;         // mapped file (refcounted via filedup)
};
```

`vmass` 배열이 프로세스마다 존재한다는 점이 설계의 출발점이다. Page Table이 프로세스마다 하나씩 존재하고, `mmap` 매핑은 해당 프로세스의 Address space 안에 존재하는 것이므로, 이 매핑을 기록하는 `vma` 역시 프로세스 단위로 분리되어야 한다. 따라서, `mmap`, `munmap`, page fault handling 등 모든 동작은 `myproc()->vmass` 를 기준으로 이루어진다.

Lazy Mapping

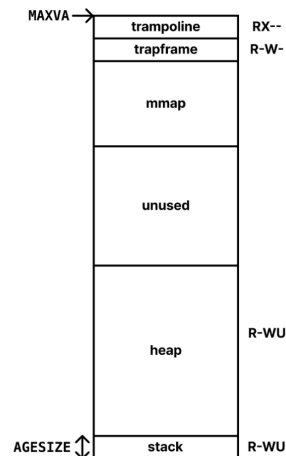
본 과제에서 가장 중요한 점은 `mmap` 이 호출되는 시점에 파일을 읽거나 physical page를 할당하는 것이 아니라는 점이다. `mmap` 은 “이 주소 범위에 이 파일을 매핑한다”라는 의도만을 VMA에 기록하고, 실제 physical page 할당과 파일 읽기는 해당 주소에 처음 접근하여 page fault가 발생할 때 한 page 단위로 수행한다.

이러한 lazy page fault를 구현하기 위해 page fault handler인 `mmapfault` 가 필요하며, `kernel/trap.c` 에 존재하는 `usertrap` 이 page fault 발생 시 `mmapfault` 를 호출하도록 이미 연결되어 있음을 확인하였다.

```
// mmapfault() only handles addresses inside an mmap VMA.
uint64 stval = r_stval();
if(vmfault(p->pagetable, stval, (r_scause() == 13) ? 1 : 0) != 0){
    // lazily-allocated heap page handled
} else if(mmapfault(stval) == 0){
    // mmap lazy fault handled
} else {
    printf("usertrap(): unexpected scause 0x%x pid=%d\n", r_scause(), p->pid);
}
```

Address Space

프로세스의 mmap 영역은 address space 상에서 TRAPFRAME 바로 아래에 위치하며 아래쪽으로 자라도록 배치한다. heap (`p->sz`)은 위쪽으로 자라므로, 두 영역이 서로 침범하지 않도록 새 매핑은 현재 사용중인 VMA들 중 가장 낮은 주소 아래에 할당하고, 그 결과 주소가 `p->sz` 를 침범하면 실패로 처리한다.



Argument Validation

과제 명세의 Step 1에서 명시된 사항으로, mmap의 모든 인자에 대해 valid한지를 검사해, 잘못된 인자에 대해서는 -1을 return하도록 설계하기로 하였다.

- `len` 은 양수이면서 `PGSIZE` 의 배수여야 한다.
- `prot` 은 READ/WRITE bit만 허용한다.
- `flags` 는 `MAP_SHARED` 와 `MAP_PRIVATE` 중 정확히 하나여야 한다.
 - 특히 `MAP_SHARED` 이면서 `PROT_WRITE` 인 매핑은 쓰기 가능한 파일에 대해서만 허용해야 한다. 이는 `munmap` 시 변경분을 파일에 write-back하는 과정에서 읽기 전용 파일에 쓰기를 시도하는 상황을 사전에 차단하기 위함이다.
 - 한편 `MAP_PRIVATE` 는 변경이 프로세스 로컬에만 남고 파일에 반영되지 않으므로, 읽기 전용 파일에 대해서도 `PROT_WRITE` 를 허용한다.

File Reference Management

VMA가 파일을 가리키는 동안에는, 사용자가 해당 File Descriptor를 `close` 하더라도 매핑이 유효하게 유지되어야 한다. 이를 위해 `mmap` 시 `filedup` 으로 파일의 참조 횟수를 증가시키고, 매핑이 완전히 제거(`munmap` 또는 프로세스 종료)될 때 `fileclose` 로 이를 되돌리도록 설계했다. 이 처리가 없으면 `fd` 가 닫히는 순간 `struct file` 이 해제되어, 이후 page fault나 write-back 과정에서 dangling pointer 문제가 발생한다.

fork & exit

`fork` 시 자식 프로세스는 부모와 동일한 매핑을 보아야 하지만 physical page를 공유하지는 않는다. 따라서 VMA 배열만 복사하고 각 파일의 참조 횟수를 증가시키며, physical page는 자식이 접근할 때 각자의 page fault를 통해 lazy하게 채우도록 한다.

`exit` 시에는 `MAP_SHARED` 매핑의 변경분이 파일에 반영되어야 하므로, 각 VMA에 대해 `munmap` 과 동일한 write-back 정리 과정을 page table이 해제되기 전에 수행한다. `munmap` 과 `exit` 의 정리 로직이 동일하므로, 이를 helper function으로 분리하여 중복을 제거하도록 설계하였다.

Git Clone 후 바로 mmaptest를 시도하면 이렇게 됨.

```

junseo@bagjunseoui-MacBookAir project3-bjsbest % make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel
o-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
test basic mmap
mmaptest: test basic mmap failed: mmap, pid=3

```

Step 1 하고 나니까 일케 됨

```

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ mmaptest
test basic mmap
usertrap(): unexpected scause 0xd pid=3
sepc=0x10c stval=0x3fffffc000

```

Implementation



Implementation

Detail the specific modifications and additions you made to the given xv6 codebase. Clearly highlight how they differ from the original code and the purpose of these changes.

Step 0. include 추가

`sys_mmap()` 함수에서 `TRAPFRAME` 매크로를 사용하기 위해, `kernel/sysfile.c` 의 include 목록에 `#include "memlayout.h"` 를 추가하였다.

```

16  #include "file.h"
17  #include "fcntl.h"
18  #include "memlayout.h"
19

```

Step 1. `sys_mmap` 함수 구현

먼저 `mmap()` 함수의 인자들을 `argint` 와 `argfd` 를 이용해 받은 후, 이 인자들이 valid한지를 다음과 같이 검증하였다. 각 인자들이 valid한지는 **Design**에서 설명한 조건을 통해 확인하였다.

```
// 1. Validate arguments.
argint(0, &len);
argint(1, &prot);
argint(2, &flags);
argint(4, &offset);
if(argfd(n: 3, pfd: &fd, pf: &f) < 0){
    return -1;
}

if(len <= 0 || (len % PGSIZE) != 0){ // len
    return -1;
}
if(prot & ~(PROT_READ | PROT_WRITE)){ // prot : allows only READ/WRITE bit.
    return -1;
}
if(flags != MAP_SHARED && flags != MAP_PRIVATE){ // flags : allows only SHARED/PRIVATE bit.
    return -1;
}
if((prot & PROT_READ) && !f->readable){ // compatible with file prot?
    return -1;
}
if((flags & MAP_SHARED) && (prot & PROT_WRITE) && !f->writable){
    return -1;
}
}
```

다음으로 이 프로세스의 vma 배열(`p->vma`)에서 `used` 가 0인 빈 slot을 찾고, 없으면 -1을 return하도록 하였다. 주소는 `TRAPFRAME` 을 기준으로, 현재 사용중인 VMA 중 가장 낮은 `addr` 을 찾아 그 아래에 `len` 만큼의 공간을 할당한다. 이 주소가 `PGROUNDUP(p->sz)` 보다 작으면 Heap section을 침범하는 것이므로 -1을 반환하도록 하였다.

```
// 2. find empty vma slot.
struct vma* v = 0;
for(int i = 0; i < NVMA; i++){
    if(!p->vmass[i].used){
        v = &p->vmass[i];
        break;
    }
}
if(v == 0)
    return -1;

// 3. select empty virtual address.
uint64 top = TRAPFRAME;
for(int i = 0; i < NVMA; i++){
    if(p->vmass[i].used && p->vmass[i].addr < top)
        top = p->vmass[i].addr;
}
uint64 addr = top - len;

if(addr < PGROUNDUP(p->sz)){ // check if addr violates heap section
    return -1;
}
}
```

마지막으로, 찾은 slot에 `addr`, `length`, `prot`, `flags`, `offset` 을 채우고, `filedup()` 함수를 통해 파일 참조 횟수 (`refcount`)를 증가시킨 뒤 할당된 주소를 return한다.

```
// 4. fill VMA, and increase refcount using filedup()
v->used = 1;
v->addr = addr;
v->length = len;
v->prot = prot;
v->flags = flags;
v->offset = offset;
v->f = filedup(f);

return addr;
```

이 단계에서는 physical page를 할당하지 않으므로 첫 mmap은 성공하지만 매핑된 주소에 접근하면 Page fault가 발생하여 프로세스가 종료된다. 이는 mmapfault가 아직 stub이기 때문이며, 의도된 동작이다.

Step 2. `mmapfault` 함수 구현

먼저 `va`를 `PGROUNDOWN`을 이용해 page의 시작 주소로 내린 뒤, `p->vmass`에서 이 주소를 포함하는 VMA를 찾는다. 찾지 못하면 매핑되지 않은 영역에 대한 접근이므로 -1을 return하며, 이 경우 `usertrap`이 프로세스를 종료시킨다. 이미 매핑된 페이지에 대해 다시 page fault가 발생한 경우, `mappages`를 다시 호출하면 `remap panic`이 발생한다. 이를 방지하기 위해 `ismapped`로 해당 주소가 이미 매핑되었는지 확인하고, 만약 이미 매핑되어 있다면 -1을 return하여 권한 위반으로 처리하도록 하였다.

```
struct proc *p = myproc();
struct vma *v = 0;

va = PGROUNDOWN(va);
for(int i = 0; i < NVMA; i++){
    struct vma *t = &p->vmass[i];
    if(t->used && va >= t->addr && va < t->addr + t->length){
        v = t;
        break;
    }
}
if(v == 0)
    return -1;

if(ismapped(p->pagetable, va))
    return -1;
```

이후에는 `kalloc`을 통해 physical page를 1 page 할당하고 `memset`을 이용해 0으로 초기화한다. 파일 내 읽을 위치는 `v->offset + (va - v->addr)`로 계산하며, `fileread`의 inode 읽기 패턴을 참고하여 `ilock` → `readi` → `iunlock` 순서로 해당 page 분량을 읽어들인다. 이때 `readi`의 두 번째 인자는 0으로 지정하여 목적지가 kernel 주소임을 명시하도록 하였다.

```
char *mem = kalloc();
if(mem == 0)
    return -1;
memset(mem, 0, PGSIZE);

struct file *f = v->f;
uint off = v->offset + (va - v->addr);

ilock(f->ip);
readi(f->ip, 0, (uint64)mem, off, PGSIZE);
iunlock(f->ip);
```

마지막으로 `prot`을 PTE 권한 bit로 변환한다. `PROT_READ` / `PROT_WRITE`와 `PTE_R` / `PTE_W`는 서로 다른 비트 값을 가지므로 명시적으로 변환해야 하며, 사용자 접근이 가능하도록 `PTE_U`를 항상 설정한다. 변환한 권한으로 `mappages`를 호출하여 physical page를 `va`에 매핑하고, 실패 시 할당한 page를 `kfree`한 뒤 -1을 return하도록 하였다. 성공 시에는 0을 return한다.

```

int perm = PTE_U;
if(v->prot & PROT_READ) perm |= PTE_R;
if(v->prot & PROT_WRITE) perm |= PTE_W;

if(mappages(p->pagetable, va, PGSIZE, (uint64)mem, perm) != 0){
    kfree(mem);
    return -1;
}

return 0;

```

Step 3. `sys_munmap` with Write-back

`sys_munmap` 은 `argaddr`, `argint` 로 주소와 길이를 받은 뒤, `p->vmas` 에서 해당 주소를 포함하는 VMA를 찾는다. 과제 명세에 따라, `munmap`의 범위는 VMA의 앞, 뒤 또는 전체여야 하며 middle hole은 거부한다. 이를 위해 `munmap`의 범위가 VMA를 벗어나는 경우와, 범위의 시작과 끝이 모두 VMA의 경계에 닿지 않는 경우를 각각 검사하여 -1을 return한다.

실제 정리 작업은 Helper function `vmaunmap` 으로 분리하여 **Step 4**의 `kexit` 에서 재사용할 수 있도록 하였다. `vmaunmap` 함수는 page 단위로 다음을 수행한다.

- `walk` 로 PTE를 확인하여, PTE가 없거나 invalid한 page (한 번도 fault가 일어나지 않은 page)는 건너뛴다.
- `MAP_SHARED` 매핑인 경우 해당 physical page의 내용을 파일로 write-back한다.
 - write-back을 할 때, `write`의 두 번째 argument는 0으로 지정하여 출발지가 kernel이 접근하는 physical address임을 명시하였다.
- 이후 `kfree` 로 page를 해제하고 PTE를 0으로 만든다.
- page 처리가 끝나면 VMA의 범위를 조정한다. 앞쪽을 잘라낸 경우 `addr` 과 `offset` 을 잘라낸 크기만큼 전진시키고, 두 경우 모두 `length` 를 줄였다. `offset` 을 함께 전진시키는 이유는 남은 영역에 다시 page fault가 발생했을 때 파일의 올바른 위치를 읽도록 하기 위함이다.
- `length`가 0이 되어 VMA가 완전히 사라지면, `fileclose` 를 통해 `mmap` 시 증가시킨 참조를 되돌리고 slot을 비우도록 하였다.

```

void vmaunmap(struct proc *p, struct vma *v, uint64 start, uint64 len){
    uint64 end = start + len;

    for(uint64 a = start; a < end; a += PGSIZE){
        pte_t *pte = walk(p->pagetable, a, 0);
        if(pte == 0 || (*pte & PTE_V) == 0)
            continue; // skip if invalid...

        if(v->flags & MAP_SHARED){
            uint64 pa = PTE2PA(*pte);
            uint off = v->offset + (a - v->addr);
            struct file *f = v->f;

            int max = ((MAXOPBLOCKS-1-1-2) / 2) * BSIZE;
            int i = 0;
            while(i < PGSIZE){
                int n1 = PGSIZE - i;
                if(n1 > max)
                    n1 = max;
                begin_op();
                ilock(f->ip);
                writei(f->ip, 0, pa + i, off + i, n1);
                iunlock(f->ip);
                end_op();
                i += n1;
            }
        }

        kfree((void*)PTE2PA(*pte));
        *pte = 0;
    }

    // modify VMA range.
    if(start == v->addr){
        v->addr += len;
        v->offset += len;
    }
    v->length -= len;

    // if length == 0 -> fclose
    if(v->length == 0){
        fclose(v->f);
        v->used = 0;
        v->addr = v->offset = v->length = v->prot = v->flags = 0;
        v->f = 0;
    }
}

```

Step 4 : kfork & kexit

kfork에서는 부모의 ofile 복사 직후, 부모의 vmas 배열 중 used 인 slot을 자식에게 그대로 복사하고 각 파일에 filedup을 수행한다. physical page는 복사하지 않으며, 자식이 해당 주소에 접근할 때 각자의 page fault로 채워진다. uvmcopy는 [0, p->sz) 범위만 복사하므로, p->sz 위에 위치한 mmap 영역은 자동으로 복사 대상에서 제외되어 별도의 처리 없이 lazy fault 방식이 유지된다.

```

// Step 4.
for(i = 0; i < NVMA; i++){
    if(p->vmass[i].used){
        np->vmass[i] = p->vmass[i];
        filedup(np->vmass[i].f);
    }
}

```

kexit에서는 File descriptor를 닫기 전에, used 인 각 VMA에 대해 Step 3에서 만든 helper function인 vmaunmap을 호출하여 정리한다. 이 위치는 page table이 해제되기 전이며, 아직 lock을 획득하지 않아 begin_op / end_op 사용이 안전한 구간이다. 이를 통해 MAP_SHARED 매핑의 변경분이 파일에 반영된 뒤 프로세스가 종료된다.

```

void
kexit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
        panic("init exiting");

    // Step 4.
    for(int i = 0; i < NVMA; i++){
        if(p->vmass[i].used)
            vmaunmap(p, &p->vmass[i], p->vmass[i].addr, p->vmass[i].length);
    }

    // Close all open files.
    for(int fd = 0; fd < NOFILE; fd++){
        if(p->ofile[fd]){

```

Results



Results

Document the compilation and execution process. Include screenshots verifying that all requirements are met, alongside explanations of the execution flow.

```
xv6 kernel is booting
```

```

hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
test basic mmap
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
sepc=0xaae stval=0x3ffffd000
usertrap(): unexpected scause 0xd pid=6
sepc=0xb28 stval=0x3ffffc000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
sepc=0xc80 stval=0x3ffffc000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded

```

```

sepc=0x46d8 stval=0x3d012000
usertrap(): unexpected scause 0xf pid=6647
sepc=0x46d8 stval=0x3e012000
usertrap(): unexpected scause 0xf pid=6648
sepc=0x46d8 stval=0x3f012000
PASS
test lazy_copy: PASS
test lazy_sbrk: PASS
PASS ALL TESTS
$

```

Implementation 이후 `mmaptest` 와 `usertests -q` 를 통해 테스트를 통과함을 확인하였다.

Troubleshooting



Troubleshooting

If you encountered any issues during the assignment, describe the issues and your resolution process. For unresolved issues, explain what they were and what approaches you attempted.

remap panic

함수 구현 중 `mmaptest` 실행 시 `panic: mappages: remap` 이 발생한 적이 있었다. 원인은 이미 매핑된 페이지에 대해 다시 page fault가 발생했을 때, `mmapfault` 가 이를 걸러내지 못하고 `mappages` 를 재차 호출한 것이었다. read-only로 매핑된 페이지에 write를 시도하면, 첫 접근에서 `PTE_R` 만으로 매핑된 뒤 쓰기 시 권한 부족으로 다시 page fault가 발생하는데, 이 두 번째 fault에서 동일한 페이지를 다시 매핑하려 하면서 panic이 발생한 것이었다.

이를 해결하기 위해 `mmapfault` 함수에서 VMA를 찾은 직후, `kalloc` 이전에 `ismapped` 로 해당 주소가 이미 매핑되었는지 검사하고, 그렇다면 -1을 return하도록 수정하였다.

```
    if(v == 0)
        return -1;

    if(ismapped(p->pagetable, va))
        return -1;

    char *mem = kalloc();
```